Price and Quality Comparison Application (PQC)

By

Ahmad Masood

Supervised By

Dr. Khalid Saleem

DEPARTMENT OF COMPUTER SCIENCE
QUAID-I-AZAM UNIVERSITY, ISLAMABAD

DEPARTMENT OF COMPUTER SCIENCE,
QUAID-I-AZAM UNIVERSITY, ISLAMABAD.

Dated: _____

FINAL APPROVAL

This is to certify that we have read the final report submitted by **Ahmad Masood** and it is our judgment that this report is of sufficient standard to warrant its acceptance by Quaid-i-Azam University, Islamabad for the degree of the **Bachelor of Science in Computer Science**.

1. **EXTERNAL EXAMINER** _____

2. **SUPERVISOR** _____

Dr. Khalid Saleem
Department of Computer Science
Quaid-i-Azam University, Islamabad.

ACKNOWLEDGEMENT

First and foremost, I bow my head in gratitude to Allah Almighty, the most beneficent and merciful, for bestowing upon me the strength, knowledge, and ability to complete this Final Year Project.

I would like to express my deep gratitude to my supervisor, Dr. Khalid Saleem, for his patient guidance, enthusiastic encouragement, and useful critiques of this research work. His insightful feedback and technical advice regarding the complexities of web scraping and system architecture were invaluable in keeping the project on track.

My special thanks go to the Department of Computer Science for providing the necessary resources and environment to complete this project. Finally, I thank my parents and friends for their prayers and moral support throughout this journey.

ABSTRACT

The **Price and Quality Comparison** Application (PQC) is a web-based information system designed to revolutionize how consumers in Pakistan discover and compare products across both online and physical retail chains. In the current fragmented market, comparing prices between local supermarkets (like Imtiaz or Punjab Cash & Carry) and online stores (like PriceOye or Daraz) is a manual and difficult process. This project addresses this gap by developing a specialized Web Scraping Engine using Puppeteer and Node.js that retrieves real-time pricing, availability, and quality ratings from target vendor websites. The system features a responsive frontend built with Next.js and utilizes Natural Language Processing (NLP) to handle Urdu-to-English translation, ensuring accessibility for a wider audience. This documentation details the software development lifecycle, including the architectural design, scraping logic, and database management required to build this centralized comparison platform.

Contents

1 Software Project Management Plan (SPMP) 1

1.1 INTRODUCTION 2

1.2 Project Overview 2

1.2.1 Motivation 2

1.2.2 Application 2

1.2.3 Existing Systems 2

1.3 Project Deliverables 3

1.4 Project Organization 3

1.4.1 Software Process Model 3

1.4.2 Roles and Responsibilities 3

1.4.3 Tools and Techniques 4

1.5 Project Management Plan 4

1.5.1 Tasks 4

1.5.2 Description 5

1.6 Gantt Chart 6

1.7 Summary 8

2 Software Requirements Specifications (SRS) 9

2.1 Introduction 10

2.2 Product Overview 10

2.3 Specific Requirements 10

2.3.1 Functional Requirements 10

2.3.2 Non-Functional Requirements 11

2.4 Software Product Features 11

2.5 Software System Attributes 12

2.6 List of Use Cases 12

2.7 Use Case Diagram 13

2.8 Use Case Descriptions 13

2.9 System Sequence Diagram 18

2.10 Activity Diagrams 21

3 Software Design Description (SDD) 26
3.1 Introduction . . . 27
3.1.1 Design Overview . . . 27
3.1.2 Design Patterns Implementation . . . 27
3.1.3 Requirements Traceability Matrix (RTM) . . . 27
3.2 System Architecture Design . . . 28
3.2.1 Architectural Overview . . . 28
3.2.2 Architectural Layers . . . 29
3.2.3 System Components Interaction . . . 29
3.3 Class Diagram . . . 30
3.4 Domain Model . . . 31

4 Software Implementation (SI) 32
4.1 System Definition . . . 33
4.2 Development Tools . . . 33
4.3 Implementation Details . . . 33
4.3.1 User Interaction Module . . . 33
4.3.2 Query Processing (NLP) Module . . . 34
4.3.3 Web Scraping and Data Retrieval Module . . . 34
4.3.4 Product Matching and Comparison . . . 34
4.4 Data Flow . . . 34
4.5 Summary . . . 34

5 Software Test Documentation (STD) 35
5.1 Introduction . . . 36
5.1.1 Test Approach . . . 36
5.2 Test Plan . . . 36
5.2.1 Testing Objectives . . . 36
5.2.2 Testing Scope . . . 36
5.2.3 Testing Tools and Environment . . . 37
5.2.4 Testing Schedule . . . 37
5.3 Test Cases . . . 37
5.3.1 Test Case: Valid Product Search . . . 37
5.3.2 Test Case: Invalid Search Input . . . 38
5.3.3 Test Case: Multilingual Query Processing . . . 38
5.3.4 Test Case: View Product Details . . . 39
5.3.5 Test Case: Product Comparison . . . 39
5.3.6 Test Case: Filter and Sort . . . 39
5.3.7 Performance Testing . . . 39

5.3.8 Reliability Testing 40
5.3.9 Test Execution Summary 40
5.4 Conclusion 40

6 Conclusion and Future Work **41**
6.1 Conclusion 42
6.2 Future Work 42

List of Tables

2.1 Use Case Description: Search Product 14
2.2 Use Case Description: View Product Details 15
2.3 Use Case Description: View Product Comparison 16
2.4 Use Case Description: Filter and Sort Products 17

3.1 Requirements Traceability Matrix 28

5.1 Testing Schedule 37
5.2 Functional Test Case: Valid Product Search 38
5.3 Functional Test Case: Invalid Search 38
5.4 Functional Test Case: Multilingual Query 38
5.5 Functional Test Case: View Product Details 39
5.6 Functional Test Case: Product Comparison 39
5.7 Functional Test Case: Filter and Sort 39
5.8 Non-Functional Test Case: Performance 40
5.9 Non-Functional Test Case: Reliability 40
5.10 Overall Test Execution Summary 40

List of Figures

1.1 Iterative Model 3
1.2 Gantt Chart 6
1.3 Gantt Chart Timeline 6
1.4 Gantt Chart 7
1.5 Gantt Chart Timeline 8

2.1 Use case Diagram 13
2.2 System Sequence Diagram: Search Product 18
2.3 System Sequence Diagram: View Product Details 19
2.4 System Sequence Diagram: View Product Comparison 20
2.5 System Sequence Diagram: Filter And Sort 21
2.6 Activity Diagram: Search Product 22
2.7 Activity Diagram: View Product Details 23
2.8 Activity Diagram: View Product Comparison 24
2.9 Activity Diagram: Filter And Sort 25

3.1 Three- Tier Architecture 28
3.2 Class Diagram 30
3.3 Domain Diagram 31

Chapter 1

Software Project Management Plan (SPMP)

1.1 INTRODUCTION

1.2 Project Overview

Price and Quality Comparison Application (PQC) is a web based application aimed at assisting. users can observe the best prices on products by comparing prices offered by different retailers. The system allows the type in of a product name (in English or Urdu), translates the query where needed and crawls the target stores (Online and Physical stores that contain online store as well) to fetch, real-time prices and ratings. The user will have an opportunity of seeing a comparison table with the best price. and quality.

1.2.1 Motivation

The primary reason why this project will come to be is because users will be able to view related products easily and compare them among the various platforms. The Pakistani retail market is characterized by the fact that consumers can hardly find alternative or similar products in the market and contrast their prices in the online marketplaces and with the physical stores that also have an online presence.

1.2.2 Application

The PQC system has diverse applications across various user demographics:

- **General Consumers:**To compare the daily grocery prices in different supermarkets such as Metro. and Imtiaz.
- **Tech Buyers:** To compare electronics specifications and prices of electronics in specialized ven- dors like Shophive and Tejar
- **Market Analysts:** Price trends can be tracked using the historical data in our database. and availability of products in the long run

1.2.3 Existing Systems

Currently, Local e-commerce sites like Daraz have their own internal search, but they do not compare their prices with competitors. Existing systems often lack:

- **Cross-Platform Comparison:** They do not compare “Physical Store” prices with “Online Store” prices.
- **Urdu Language Support:** Most platforms strictly require English input.
- **Unit Normalization:** They fail to compare “1kg” with “1000g” accurately.

1.3 Project Deliverables

The following are the key deliverables of the PQC project:

- Fully functional web application built on Next.js for searching, filtering, and viewing product comparisons.
- Backend scraping API to extract data from Books, Electronics, and Grocery websites.
- MongoDB database storing product data and user search logs.
- Complete project documentation including system architecture, code details, and user manuals.

1.4 Project Organization

1.4.1 Software Process Model

The PQC Application will be developed under an Iterative and Incremental software. process model. Considering that web scraping is a complicated process because targeted sites are often altered. alter their design), one category to construct the scraper (e.g. Books) initially, try out, and then increase to others (Electronics, Grocery). This ensures continuous feedback and stability.

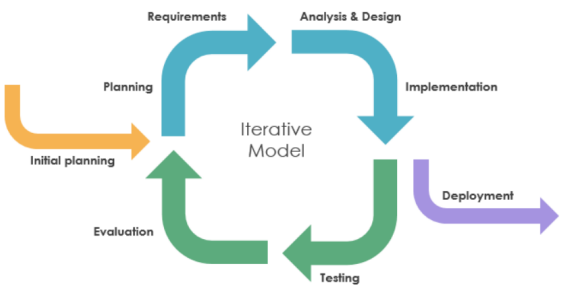

Figure 1.1: Iterative Model

1.4.2 Roles and Responsibilities

Student:

- Full stack development using Next.js for frontend and Node.js for backend APIs
- Web scraping using Puppeteer for dynamic content handling
- MongoDB database design and query optimization

- Testing and validation of scraped data accuracy
- Verification of Urdu-to-English translation functionality
- Debugging, performance optimization, and documentation

Supervisor:

- Project guidance and supervision
- Review of project progress and deliverables
- Ensuring academic and technical standards

1.4.3 Tools and Techniques

- **Development Environment:** Visual Studio Code (VS Code).
- **Frontend Framework:** Next.js (React) for a responsive UI.
- **Backend Runtime:** Node.js for handling server-side logic.
- **Automation Tool:** Puppeteer (for headless browser scraping).
- **Database:** MongoDB for flexible product data storage.
- **Version Control:** Git and GitHub.

1.5 Project Management Plan

This section outlines the planning, main tasks, and phase-wise activities of the Product Comparison System.

1.5.1 Tasks

The project is divided into the following high-level tasks:

1. Requirement Gathering and Store Selection
2. System Architecture Design
3. Development of Scraping Modules
4. Frontend Development and Integration
5. Testing and Validation
6. Final Documentation

1.5.2 Description

Requirements and Analysis Phase

This phase focused on identifying essential product attributes for comparison (e.g., title, price, rating, availability) and selecting target websites.

Activities:

- Analyzing website structures.
- Defining quality rating and unit conversion logic.
- Finalizing the tech stack and libraries.

Design Phase

The design phase created the system blueprint and workflow.

Activities:

- Designing system architecture and ERD.
- Creating wireframes for search and comparison pages.
- Flowcharting the scraping and data processing pipeline.

Development Phase:

Implemented core system functions and integrated frontend components.

Testing and Validation Phase

Ensured data accuracy, correct comparison results, and smooth system operation.

1.6 Gantt Chart

		Name	Duration	Start	Finish	Predecessors
1		**Project Planning**	**87 days?**	**11/27/25, 8:00 AM**	**3/27/26, 5:00 PM**	
2		Process	3 days?	11/27/25, 8:00 AM	12/1/25, 5:00 PM	
3		**Software Project Management Plan (SPMP)**	**11 days?**	**11/27/25, 8:00 AM**	**12/11/25, 5:00 PM**	
4		**Introduction**	**2 days?**	**11/27/25, 8:00 AM**	**11/28/25, 5:00 PM**	
5		**Project overview**	**1 day?**	**11/27/25, 8:00 AM**	**11/27/25, 5:00 PM**	
6		Motivation	1 day?	11/27/25, 8:00 AM	11/27/25, 5:00 PM	
7		Application	1 day?	11/27/25, 8:00 AM	11/27/25, 5:00 PM	
8		Existing Systems	1 day?	11/27/25, 8:00 AM	11/27/25, 5:00 PM	
9		Project Deliverables	2 days?	11/27/25, 8:00 AM	11/28/25, 5:00 PM	
10		**Project Organization**	**6 days?**	**12/1/25, 8:00 AM**	**12/8/25, 5:00 PM**	**4**
11		Software Process Model	3 days?	12/1/25, 8:00 AM	12/3/25, 5:00 PM	
12		Roles and Responsibilities	1 day?	12/4/25, 8:00 AM	12/4/25, 5:00 PM	11
13		Tools and Techniques	2 days?	12/5/25, 8:00 AM	12/8/25, 5:00 PM	12
14		**Project Management Plan**	**3 days?**	**12/9/25, 8:00 AM**	**12/11/25, 5:00 PM**	**10**
15		Tasks	1 day?	12/9/25, 8:00 AM	12/9/25, 5:00 PM	
16		Description	2 days?	12/10/25, 8:00 AM	12/11/25, 5:00 PM	15
17		Summary	1 day?	12/9/25, 8:00 AM	12/9/25, 5:00 PM	
18		M1:MileStone Completes	0 days?	12/11/25, 5:00 PM	12/11/25, 5:00 PM	16
19		**Software Requirements Specifications (SRS)**	**37 days?**	**12/12/25, 8:00 AM**	**2/2/26, 5:00 PM**	**18**
20		Introduction	1 day?	12/12/25, 8:00 AM	12/12/25, 5:00 PM	
21		Overview	1 day?	12/12/25, 8:00 AM	12/12/25, 5:00 PM	
22		**Requirements**	**4 days?**	**12/12/25, 8:00 AM**	**12/17/25, 5:00 PM**	
23		Functional Requirements	2 days?	12/12/25, 8:00 AM	12/15/25, 5:00 PM	
24		Non-Functional Requirements	2 days?	12/16/25, 8:00 AM	12/17/25, 5:00 PM	23
25		Software Product Features	1 day?	12/12/25, 8:00 AM	12/12/25, 5:00 PM	
26		Software System Attributes	1 day?	12/12/25, 8:00 AM	12/12/25, 5:00 PM	
27		List of Use Cases	1 day?	12/12/25, 8:00 AM	12/12/25, 5:00 PM	
28		Use Case Diagram	7 days?	12/12/25, 8:00 AM	12/22/25, 5:00 PM	
29		Use Case description	9 days	12/23/25, 8:00 AM	1/2/26, 5:00 PM	28
30		System sequence Diagram	13 days?	1/5/26, 8:00 AM	1/21/26, 5:00 PM	29
31		Activity Diagram	8 days?	1/22/26, 8:00 AM	2/2/26, 5:00 PM	30
32		M2:MileStone Completes	0 days?	2/2/26, 5:00 PM	2/2/26, 5:00 PM	31
33		**Software Design Description**	**11 days?**	**2/3/26, 8:00 AM**	**2/17/26, 5:00 PM**	**32**

FYP-1 - page1

Figure 1.2: Gantt Chart

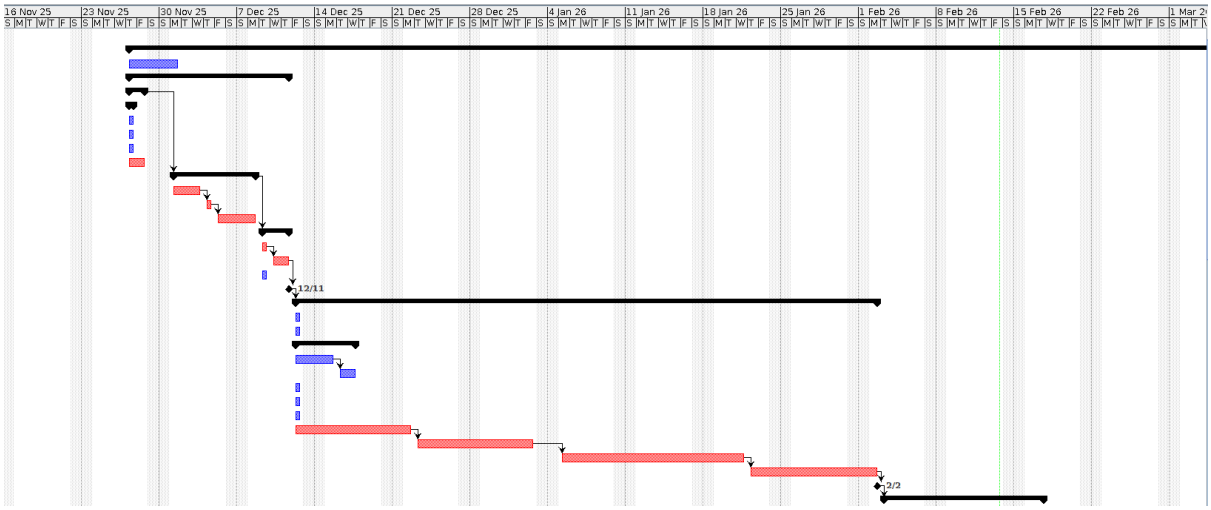

Figure 1.3: Gantt Chart Timeline

33		⊟**Software Design Description**	11 days?	**2/3/26, 8:00 AM**	**2/17/26, 5:00 PM**	**32**	
34		⊟**Introduction**	2 days?	**2/3/26, 8:00 AM**	**2/4/26, 5:00 PM**		
35		Design Overview	1 day?	2/3/26, 8:00 AM	2/3/26, 5:00 PM		
36		Design Patterns Implementation	1 day?	2/3/26, 8:00 AM	2/3/26, 5:00 PM		
37		Requirements Traceability Matrix	2 days?	2/3/26, 8:00 AM	2/4/26, 5:00 PM		
38		⊟**System Architecture Design**	2 days?	**2/5/26, 8:00 AM**	**2/6/26, 5:00 PM**	**34**	
39		Architectural Overview	1 day?	2/5/26, 8:00 AM	2/5/26, 5:00 PM		
40		Architectural Layers	2 days?	2/5/26, 8:00 AM	2/6/26, 5:00 PM		
41		System Components Interaction	1 day?	2/5/26, 8:00 AM	2/5/26, 5:00 PM		
42		Class Diagram	4 days?	2/9/26, 8:00 AM	2/12/26, 5:00 PM	38	
43		Domain Modal	3 days?	2/13/26, 8:00 AM	2/17/26, 5:00 PM	42	
44		M3:MileStone Completes	0 days?	2/17/26, 5:00 PM	2/17/26, 5:00 PM	43	
45		⊟**Software Implementation (SI)**	10 days?	**2/18/26, 8:00 AM**	**3/3/26, 5:00 PM**	**44**	
46		System Definition	1 day?	2/18/26, 8:00 AM	2/18/26, 5:00 PM		
47		Development Tools	1 day?	2/18/26, 8:00 AM	2/18/26, 5:00 PM		
48		⊟**Implementation Details**	8 days	**2/18/26, 8:00 AM**	**2/27/26, 5:00 PM**		
49		User Interaction Module	2 days	2/18/26, 8:00 AM	2/19/26, 5:00 PM		
50		Query Processing (NLP) Module	2 days	2/20/26, 8:00 AM	2/23/26, 5:00 PM	49	
51		Web Scraping and Data Retrieval Module	2 days	2/24/26, 8:00 AM	2/25/26, 5:00 PM	50	
52		Product Matching and Comparison Engine	2 days	2/26/26, 8:00 AM	2/27/26, 5:00 PM	51	
53		Data Flow	1 day?	3/2/26, 8:00 AM	3/2/26, 5:00 PM	48	
54		Summary	1 day?	3/3/26, 8:00 AM	3/3/26, 5:00 PM	53	
55		M4:MileStone Completes	0 days?	3/3/26, 5:00 PM	3/3/26, 5:00 PM	54	
56		⊟**Software Test Documentation (STD)**	12 days?	**3/4/26, 8:00 AM**	**3/20/26, 8:00 AM**	**55**	
57		⊟**Introduction**	1 day?	**3/4/26, 8:00 AM**	**3/4/26, 5:00 PM**		
58		Test Approach	1 day?	3/4/26, 8:00 AM	3/4/26, 5:00 PM		
59		⊟**Test Plan**	8 days	**3/5/26, 8:00 AM**	**3/16/26, 5:00 PM**	**57**	
60		Testing Objectives	2 days	3/5/26, 8:00 AM	3/6/26, 5:00 PM		
61		Testing Scope	2 days	3/9/26, 8:00 AM	3/10/26, 5:00 PM	60	
62		Testing Tools and Environment	2 days	3/11/26, 8:00 AM	3/12/26, 5:00 PM	61	
63		Testing Schedule	2 days	3/13/26, 8:00 AM	3/16/26, 5:00 PM	62	
64		Test Cases	2 days	3/17/26, 8:00 AM	3/18/26, 5:00 PM	59	
65		Conclusion	1 day?	3/19/26, 8:00 AM	3/19/26, 5:00 PM	64	
66		M5:MileStone Completes	0 days	3/20/26, 8:00 AM	3/20/26, 8:00 AM	65	
67		⊟**Conclusion and Future Work**	4 days	**3/20/26, 8:00 AM**	**3/26/26, 8:00 AM**	**66**	
68		Conclusion	2 days	3/20/26, 8:00 AM	3/23/26, 5:00 PM		
69		Future Work	2 days	3/24/26, 8:00 AM	3/25/26, 5:00 PM	68	
70		M6:MileStone Completes	0 days	3/26/26, 8:00 AM	3/26/26, 8:00 AM	69	

Figure 1.4: Gantt Chart

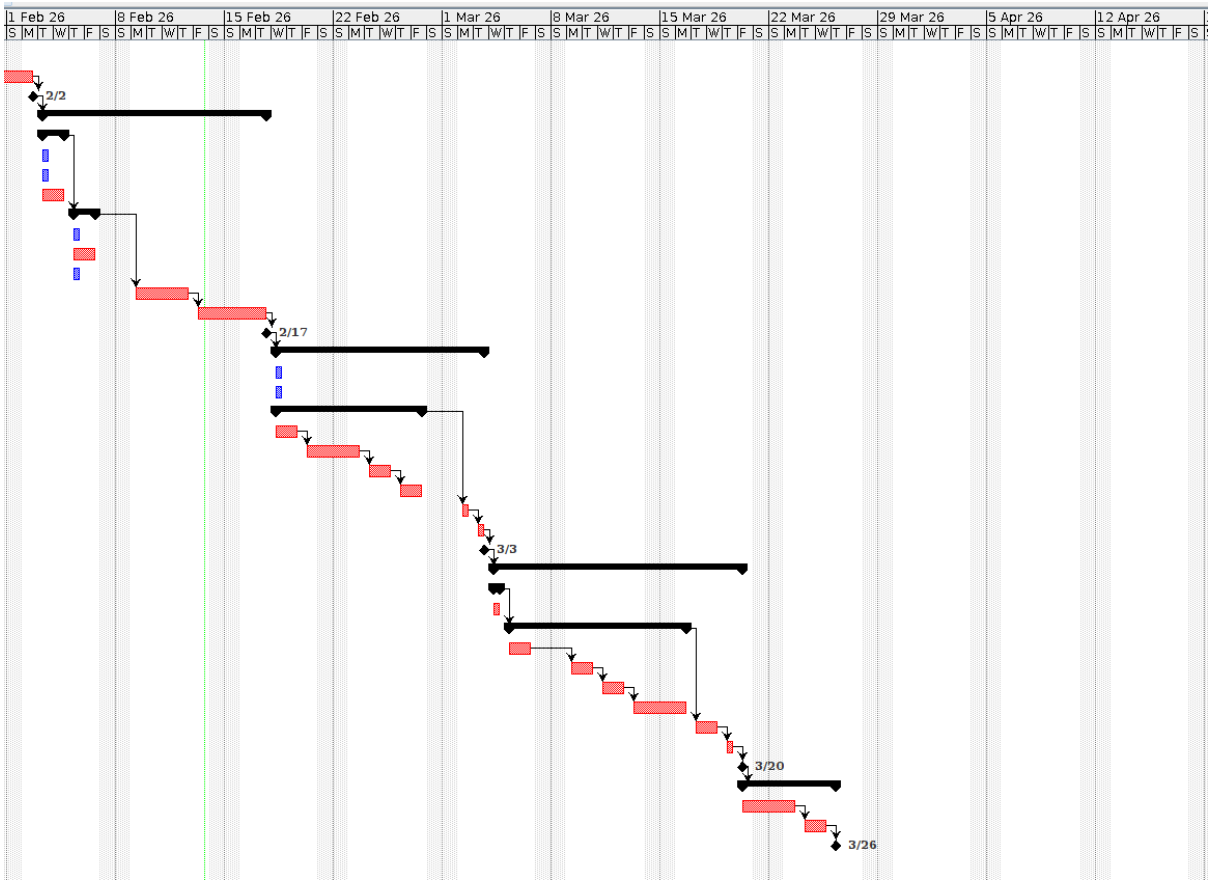

Figure 1.5: Gantt Chart Timeline

1.7 Summary

This chapter presented an overview of the Product Comparison System, including its objectives, motivation, and application scope. The development methodology, project tasks, and management plan were discussed in detail.

Chapter 2

Software Requirements Specifications (SRS)

2.1 Introduction

The chapter will offer a brief description of the Product Price and Quality Comparison System , with an emphasis on the system design, requirements, and functionality. It provides a structured description of the software product, its requirements, features, and system attribute . Also, the chapter gives the use cases and system sequence diagrams, which taken together show the interaction between users, system and outside parties like the Web scraping engine and NLP Module.It aimed for defining the system.

2.2 Product Overview

The Price and Quality Comparison System is a web based system that allow users to search the products, compare prices,rating among various products.It NLP component to decipher the user query and a Web Scraping Engine to obtain real-time information of various online and physical databases. The users may either search by the name or description of the item or see a detailed product. The system aggregates data to offer a convenient side by side price and quality comparison

The stakeholders are:

- **End Users (Shoppers):** People who shop are the ones using it to look up items, check differences and visit the products.
- **System Administrators:** In charge of the scraping engine configurations, health monitoring of the system, and control of the product catalog cache.
- **Retailers (Data Sources):** TThe online and physical stores, the data of whose products (prices, rating). The system crawls and compares description, and quality ratings.

2.3 Specific Requirements

The system requirements are categorized into functional and non-functional requirements:

2.3.1 Functional Requirements

1. **Product Search**
    - The system must allow users to enter a product name or description.
    - The system must process queries using an NLP module to handle translation and normalization.
    - The system must crawl online sources via a Web Scraping Engine to retrieve data.

2. **Product Comparison**
    - The system must compile data to compare prices and quality across sources.
    - Comparison data must only be displayed when complete data from all sources is available.
3. **Data Filtering and Sorting**
    - Users must be able to filter results by category, price, or brand.
    - Users must be able to sort results by price (low-to-high) or rating.
4. **Product Details**
    - The system must display detailed information when a user selects a specific product.
    - The system must utilize cached results to display details efficiently.

2.3.2 Non-Functional Requirements

1. **Performance Requirements**
    - The system must provide real-time updates from sources.
    - Filtering and sorting must be processed in real-time.
    - System response time should be minimal to ensure a smooth user experience.
2. **Reliability Requirements**
    - The system should handle network failures gracefully without crashing.
    - If sources fail to respond, the system must mark them as unavailable rather than failing completely.
3. **Usability Requirements**
    - The NLP module should handle multilingual queries (e.g., Urdu to English).
    - Comparison results must be clear and easy to understand.
    - Error messages must be user-friendly (e.g., ”Product details unavailable”).

2.4 Software Product Features

1. **Intelligent Search Processing:** Utilization of NLP for query translation and normalization.

2. **Multi-Source Scraping:** Aggregation of product data from various online and physical store APIs.
3. **Automated Comparison:** Side-by-side comparison of product price and quality.
4. **Dynamic Filtering:** Real-time refinement of search results based on user criteria.
5. **Caching Mechanism:** Efficient storage of search results to speed up detailed views.

2.5 Software System Attributes

1. **Reliability:** Handles missing data sources or network failures without system collapse.
2. **Performance:** Optimized for fast retrieval of cached details and real-time sorting.
3. **Usability:** Specialized handling of multilingual inputs and clear error prompting.

2.6 List of Use Cases

The primary use cases of the system are:

1. Search Product
2. View Product Details
3. View Product Comparison
4. Filter and Sort Products

2.7 Use Case Diagram

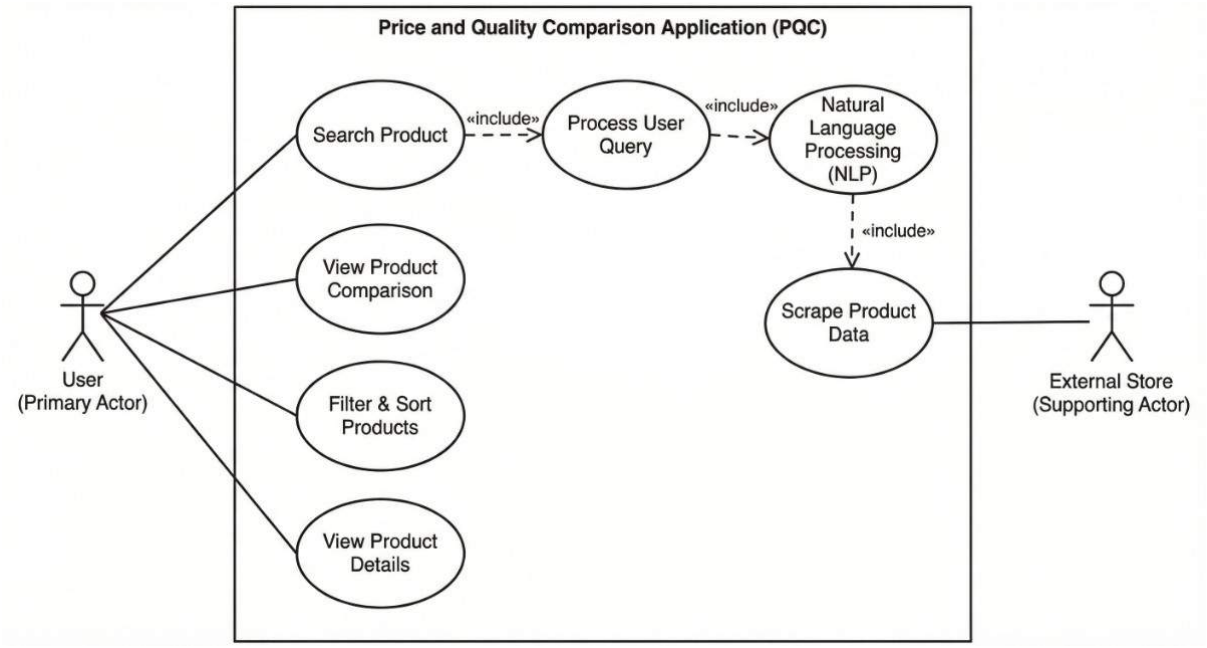

Figure 2.1: Use case Diagram

2.8 Use Case Descriptions

Use Case ID	UC-01
Use Case Name	Search Product
Primary Actor	User
Description	This allows the user to search for a product by entering its name.
Preconditions	• System has access to online store APIs or scraping modules.
Postconditions	The user is presented with a list of products matching their search query, including prices, quality information.

Main Success Scenario	1. User enters the product name or description in the search bar. 2. System sends the input to the NLP module for query processing. 3. NLP module normalizes the query. 4. System send the processed query to the Web Scraping Engine. 5. Web Scraping Engine retrieves relevant product data from various online. 6. System compiles the product data. 7. System displays the search results in a user-friendly interface.
Alternative Scenarios	• **1a. Invalid Query:** User have to enter a valid product name. • **2a. NLP Failure:** System prompts user to rephrase query. • **5a. No Products Found:** System shows ”No products found”.
Frequency	High, whenever a user wants to search and compare products.

Table 2.1: Use Case Description: Search Product

Use Case ID	UC-02
Use Case Name	View Product Details
Primary Actor	User
Description	A user views detailed information of a product after selecting it from the search results.
Preconditions	• Search results have already been displayed to the user. • User has selected a product from the search results.
Postconditions	Product details are displayed to the user
Main Success Scenario	1. System displays the search results to the user. 2. User clicks on a product from the search results. 3. System checks whether the product details are available. 4. Product details are found in the DB. 5. System displays the product details to the user.
Alternative Scenarios	• **3a. Product details not available in found:** System displays an error message: ”Product details unavailable.”
Frequency	High; whenever a user selects a product from search results.

Table 2.2: Use Case Description: View Product Details

Use Case ID	UC-03
Use Case Name	View Product Comparison
Primary Actor	User
Description	A user to view a comparison of a selected product across multiple sources.
Preconditions	• User has searched for a product. • Product comparison data has already been collected by the system.
Postconditions	If complete data is available, product comparison is displayed. If data is not available, no comparison is shown.
Main Success Scenario	1. User selects a product from the product results. 2. System receives the selected product information and comparison data. 3. System verifies that comparison data is complete for all sources. 4. System displays the product comparison to the user.
Alternative Scenarios	• **4a. Comparison Data Not Available:** If data is incomplete, the system does not display the comparison .
Frequency	Medium

Table 2.3: Use Case Description: View Product Comparison

Use Case ID	UC-04
Use Case Name	Filter and Sort Products
Primary Actor	User
Description	The user to refine the product list by applying filter criteria (e.g., category, price) and sorting options.
Preconditions	• The user has performed a product search.
Postconditions	Filtered or sorted products are displayed.
Main Success Scenario	1. User views the product search results. 2. User selects one or more filter options. 3. User selects a sorting option. 4. System applies the selected filter criteria. 5. System sorts the filtered product list. 6. System displays the updated filtered and sorted products.
Alternative Scenarios	• **3a. User Selects Only Sorting:** System sorts complete list without filters. • **4a. No Products Match Filter:** System displays ”No matching products”
Frequency	Low; Frequently used after performing a product search.
Special Requirements	• Filters must be easy to apply and remove.

Table 2.4: Use Case Description: Filter and Sort Products

2.9 System Sequence Diagram

Search Product:

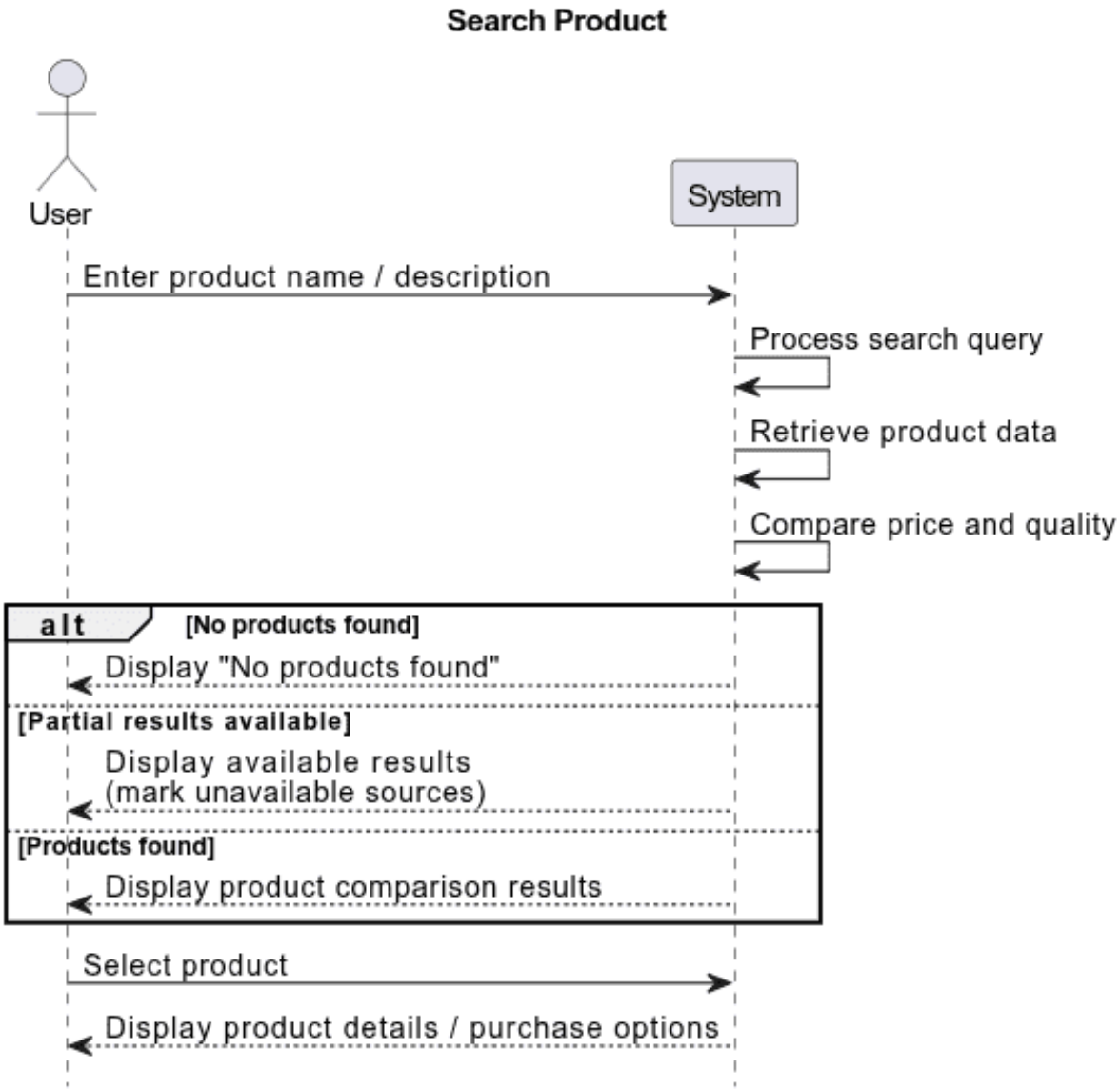

Figure 2.2: System Sequence Diagram: Search Product

View Product Details:

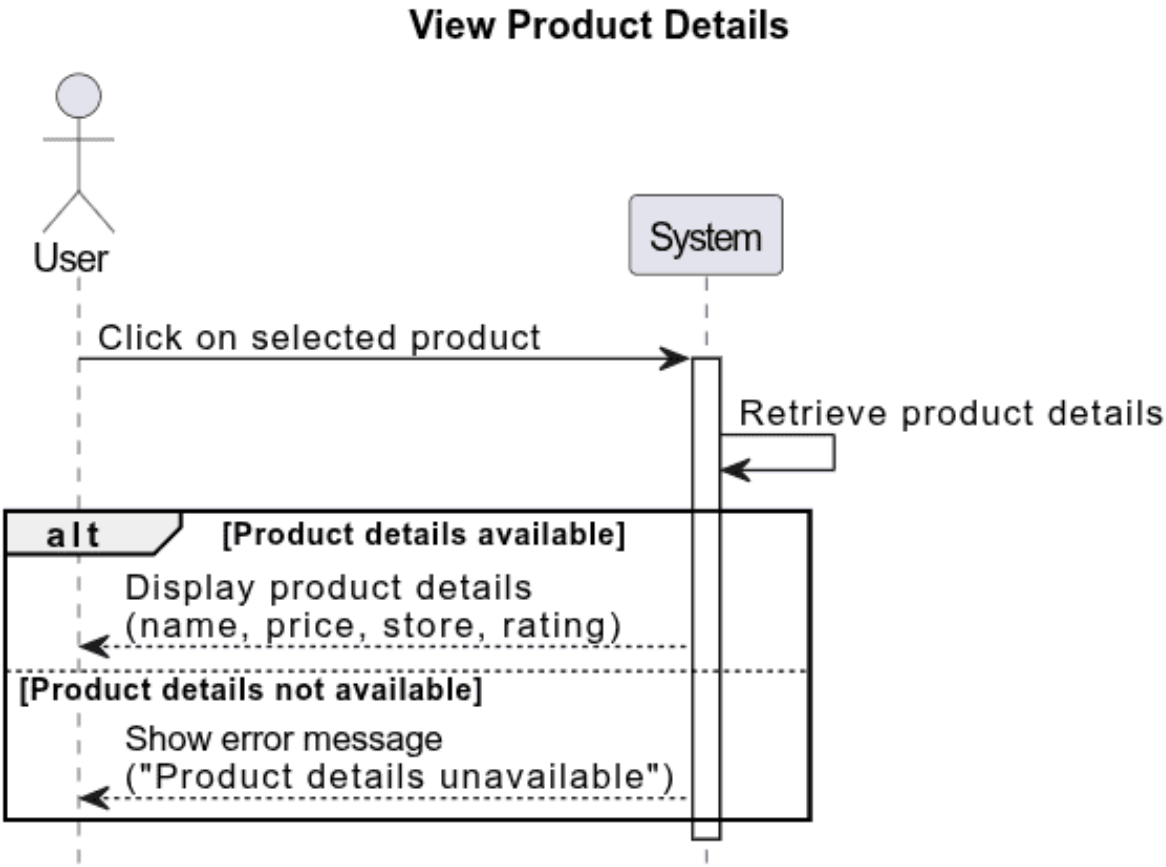

Figure 2.3: System Sequence Diagram: View Product Details

View Product Comparison:

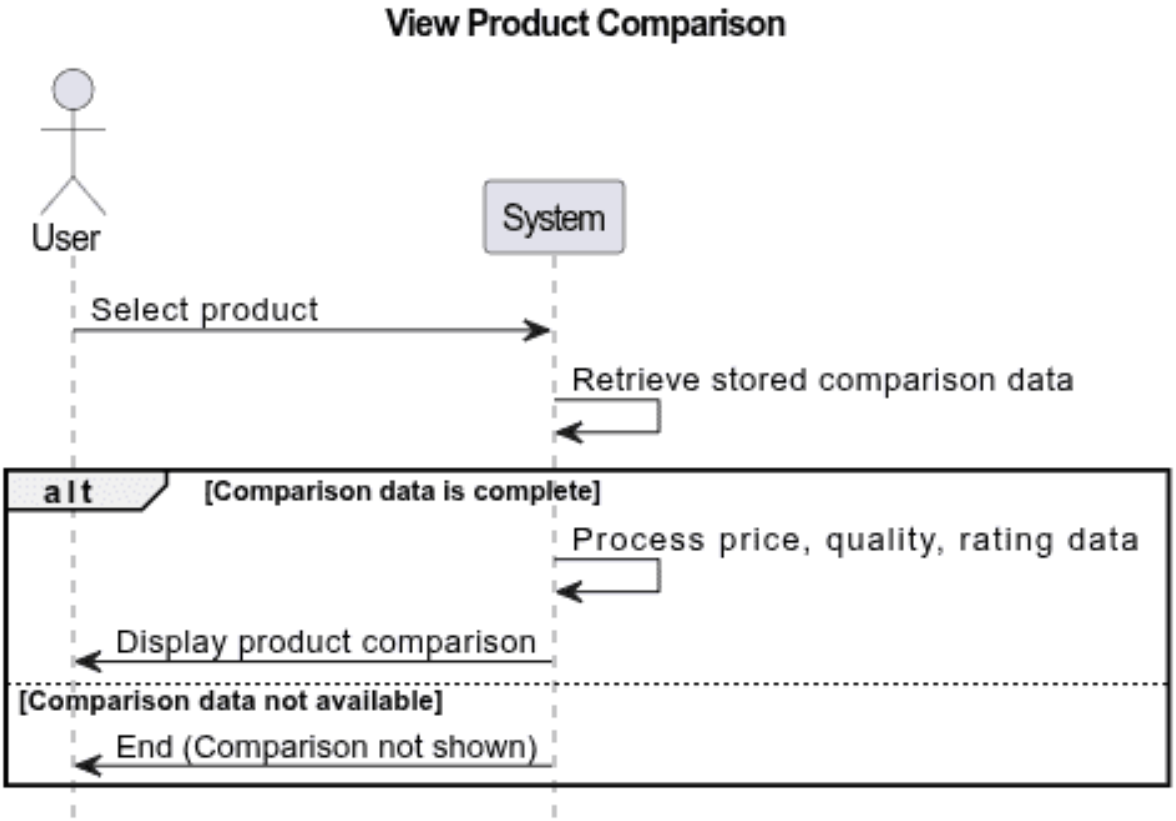

Figure 2.4: System Sequence Diagram: View Product Comparison

Filter And Sort:

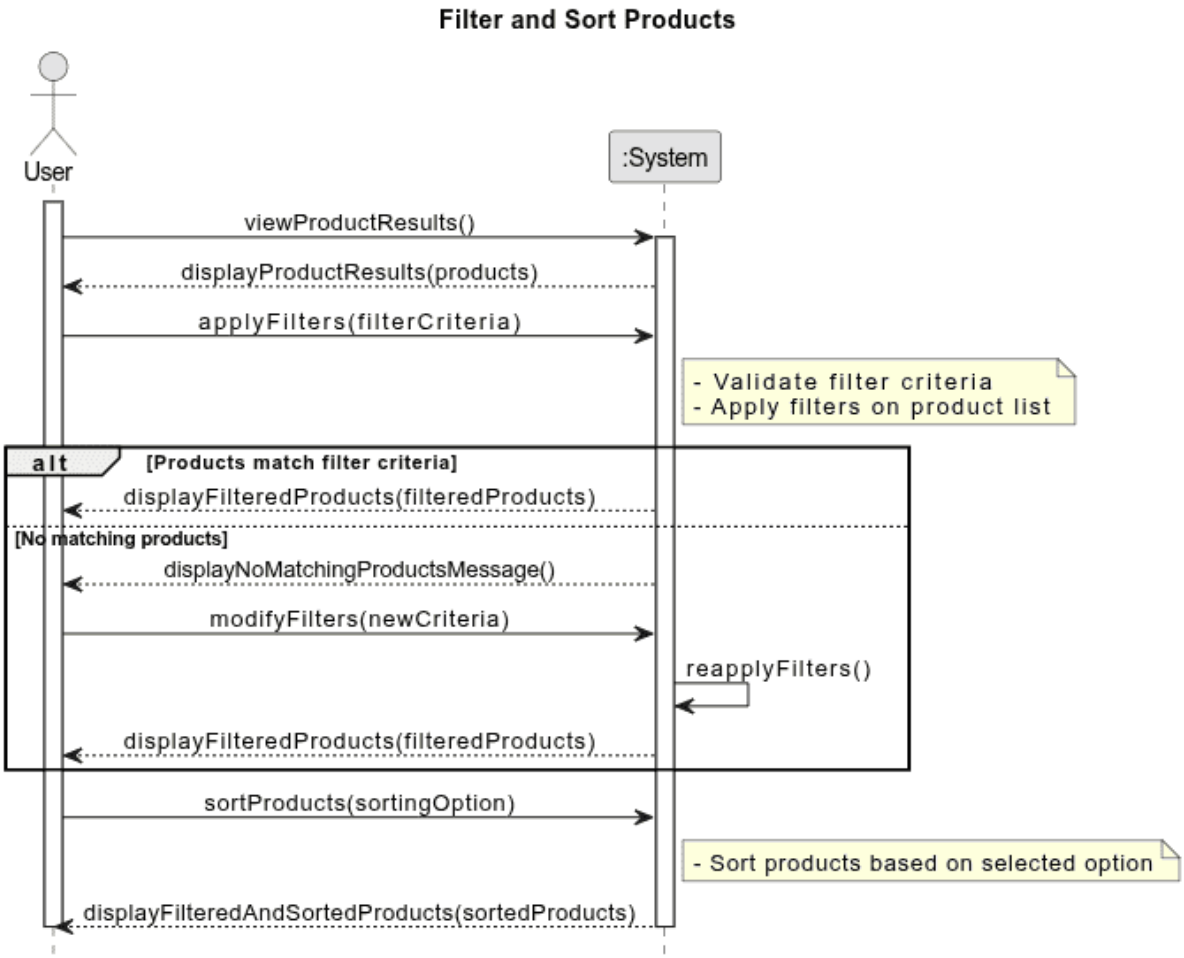

Figure 2.5: System Sequence Diagram: Filter And Sort

2.10 Activity Diagrams

1. Search Product Workflow
2. View Product Details
3. View Product Comparison
4. Filter and Sort Products Logic

Search Product:

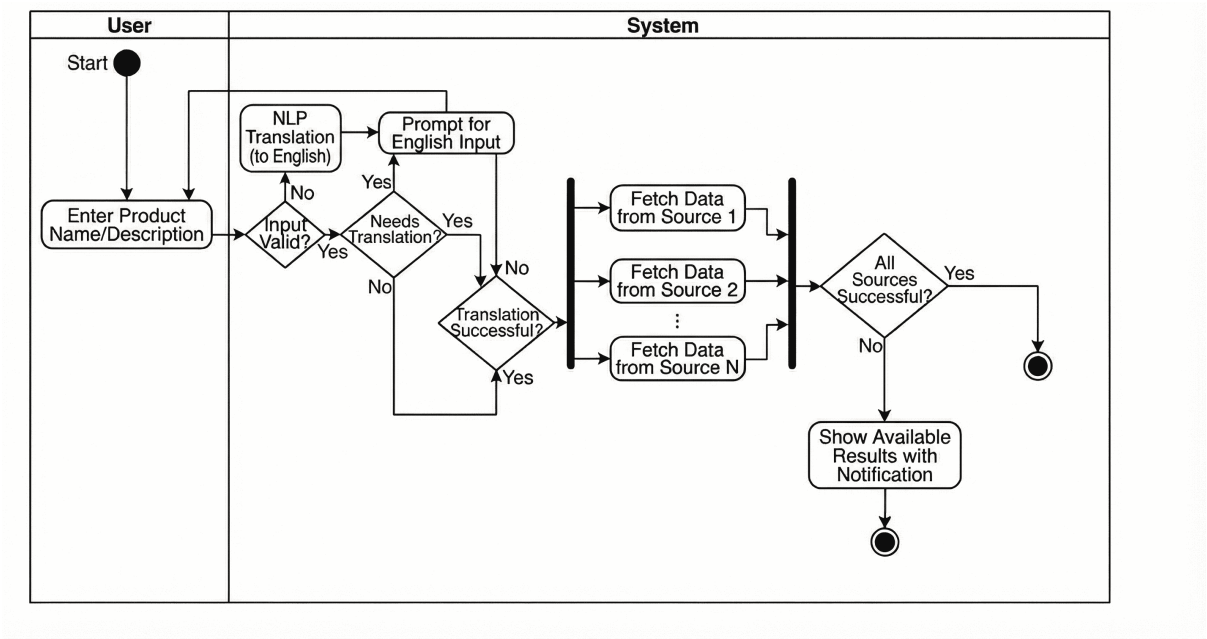

Figure 2.6: Activity Diagram: Search Product

View Product Details:

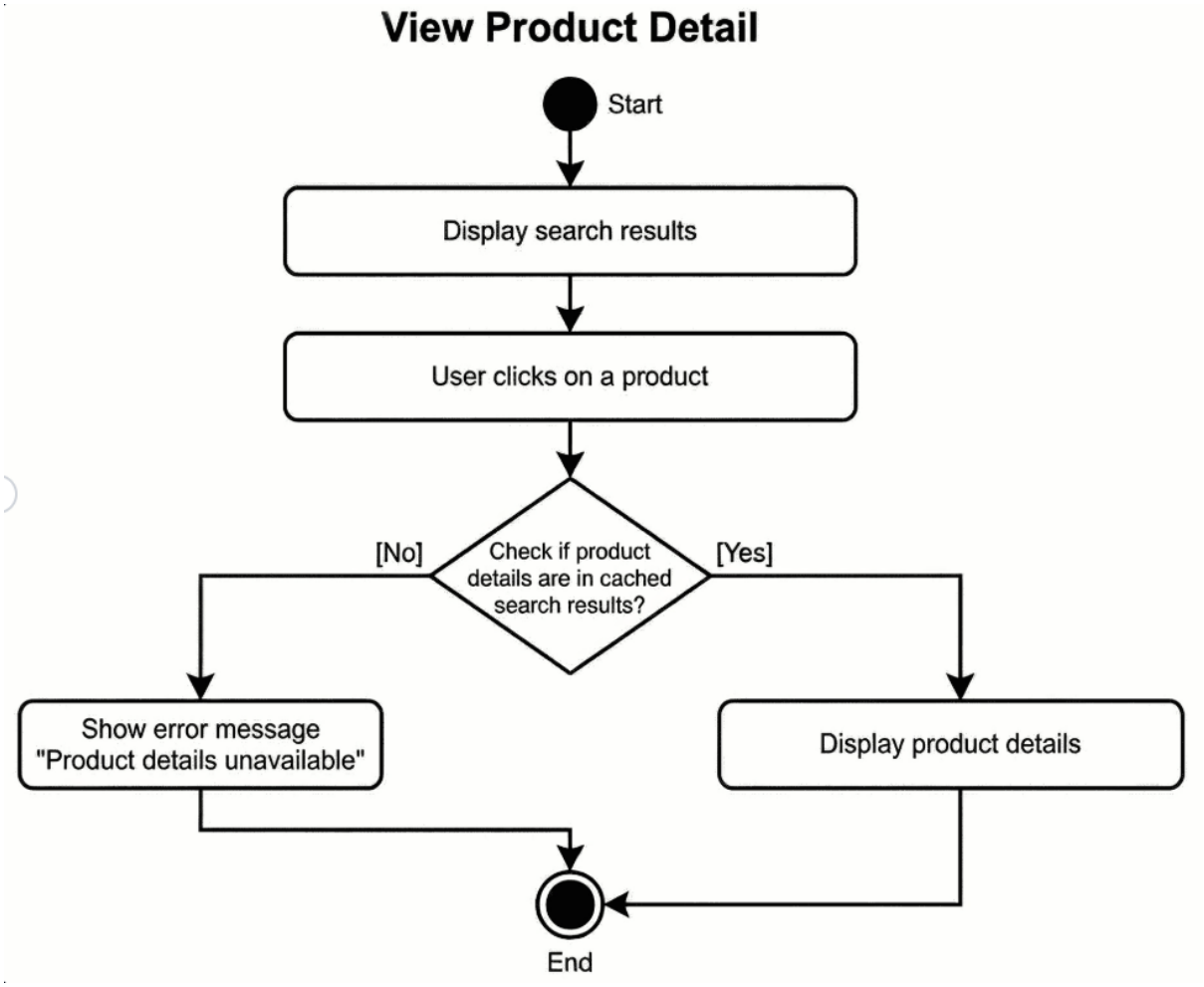

Figure 2.7: Activity Diagram: View Product Details

View Product Comparison:

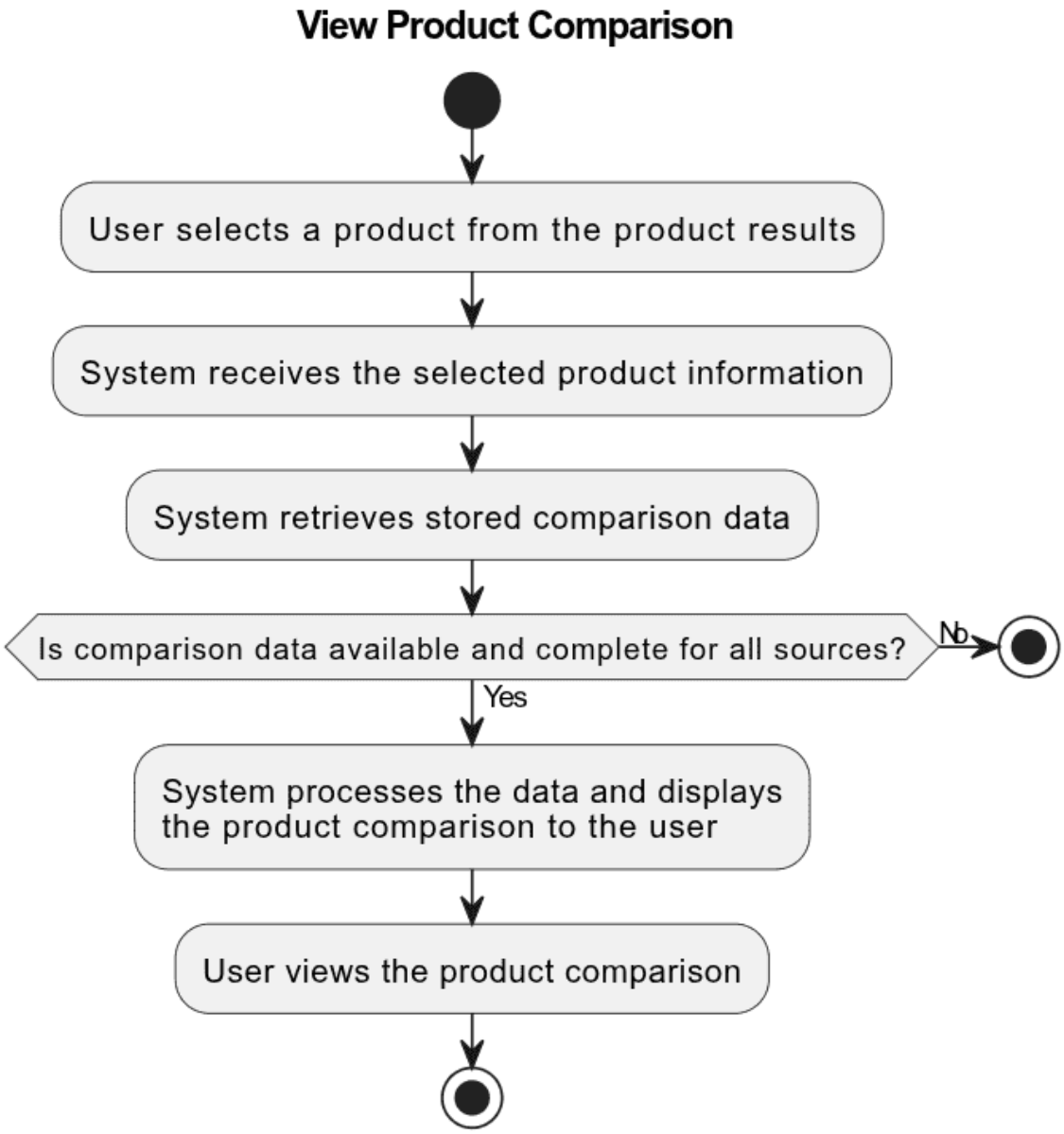

Figure 2.8: Activity Diagram: View Product Comparison

Filter And Sort:

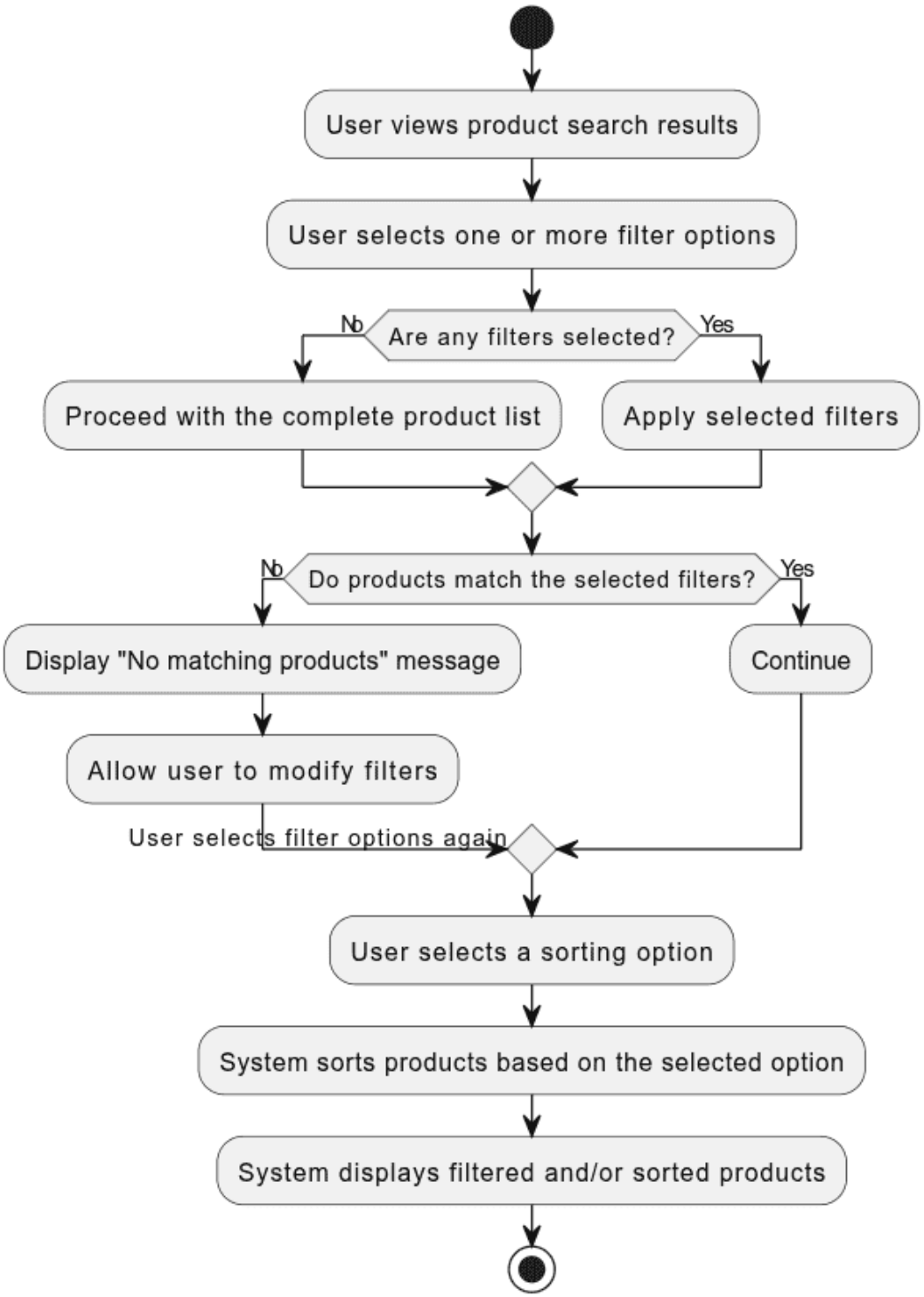

Figure 2.9: Activity Diagram: Filter And Sort

Chapter 3

Software Design Description (SDD)

3.1 Introduction

This chapter provides a comprehensive overview of the design and architectural aspects of the Product Price and Quality Comparison System. The design phase translates the functional and non-functional requirements identified in Chapter 2 into a concrete system blueprint.

3.1.1 Design Overview

This project is a modular web-based **Product Price and Quality Comparison System (PQC)** developed using a modern full-stack approach

- The **Next.js (React)** for the frontend to ensure a responsive and interactive user interface.
- The backend is powered by **Node.js** and specifically handling the **Natural Language Processing (NLP)** and **Web Scraping** modules.
- **MongoDB** is used as the primary database to store user logs, cached product details, and comparison history.

3.1.2 Design Patterns Implementation

- **Model-View-Controller (MVC):** To separate the product data models, the user interface views, and the control logic for scraping and filtering.
- **Strategy Pattern:** Used within the Web Scraping Engine to handle different scraping strategies for various e-commerce websites.

3.1.3 Requirements Traceability Matrix (RTM)

Requirement ID	**Use Case / Requirement**	**Mapped Test Cases**	**Sequence Diagram**
UC-01	Search Product (NLP & Scraping)	TC-FUNC-01, TC-FUNC-02, TC-NLP-01	SSD-01
UC-02	View Product Details	TC-FUNC-03	SSD-02
UC-03	View Product Comparison	TC-COMP-01	SSD-03
UC-04	Filter & Sort Products	TC-FILT-01	SSD-04
NFR-01	Performance (Response Time ¡ 3 sec)	TC-PERF-01	SSD-01

NFR-02	Reliability (Source Failure Handling)	TC-REL-01	SSD-01

Table 3.1: Requirements Traceability Matrix

3.2 System Architecture Design

This section describes the architectural structure of the system, including its major components and their relationships.

3.2.1 Architectural Overview

The system follows a client-server architecture where the client handles user interaction and the server manages application logic and data processing. The architecture is designed to support concurrent users, efficient data handling, and secure communication between components.

- Scalability to support increasing user requests
- Maintainability through modular design
- Reliability and fault tolerance

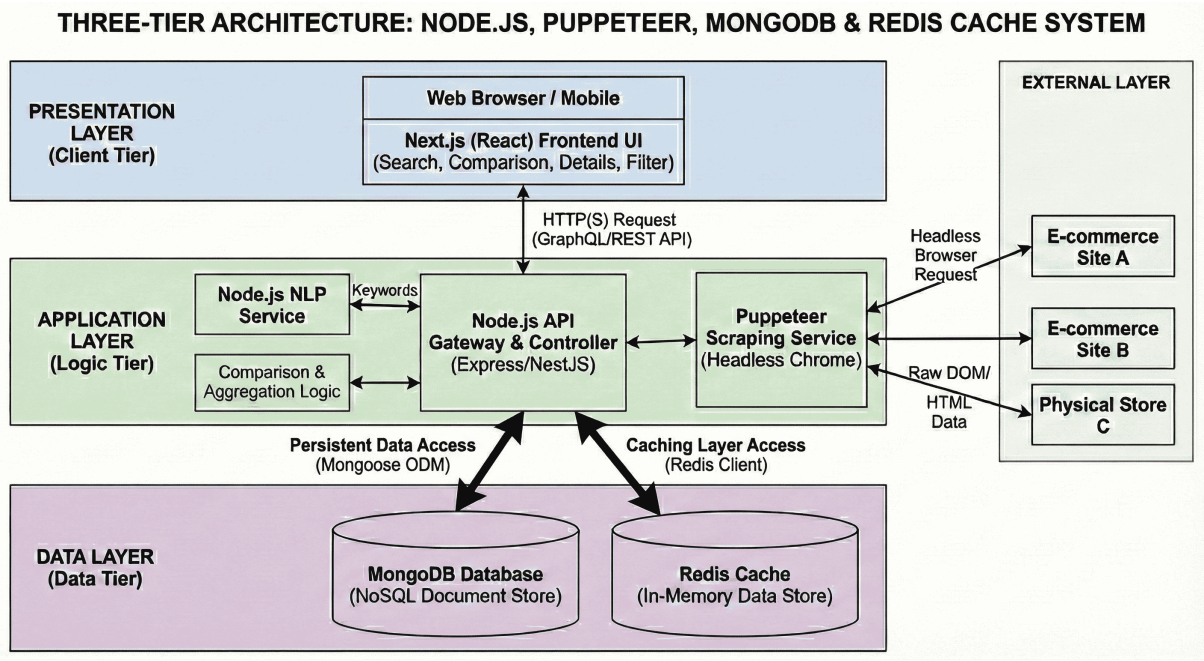

Figure 3.1: Three- Tier Architecture

3.2.2 Architectural Layers

1. **Presentation Layer**
   - Responsible for user interaction
   - Displays system output and collects user input
   - Ensures a responsive and user-friendly interface
2. **Application Layer**
   - Contains the core business logic
   - Processes user requests
   - Coordinates between the presentation and data layers
3. **Data Layer**
   - Manages data storage and retrieval
   - Ensures data consistency and integrity
   - Handles database operations

3.2.3 System Components Interaction

1. The process begins in the **Presentation Layer**, where the user initiates a request.
2. The request is forwarded to the **Application Layer**, which contains the business logic and coordinates the necessary operations.
3. If data access is required, the Application Layer communicates with the **Data Layer** to perform storage or retrieval operations.
4. Processed results flow back through the Application Layer to the Presentation Layer.
5. Finally, the Presentation Layer renders the response and displays it to the user.

This interaction model provides the following advantages:

- **HTTP/REST APIs:** Communication between Presentation and Application layers
- **ORM Queries:** Communication between Application and Data layers
- **Internal Service Calls:** Communication between application components

3.3 Class Diagram

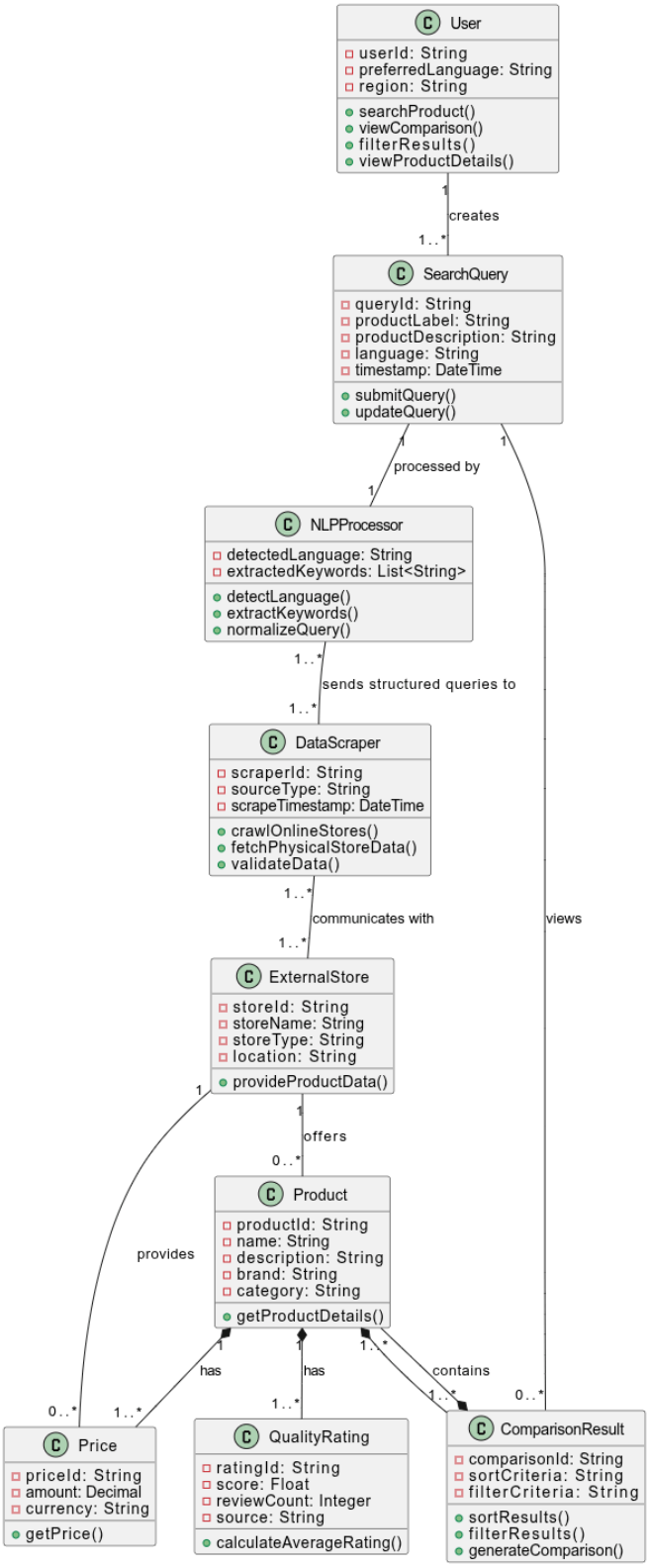

Figure 3.2: Class Diagram

3.4 Domain Model

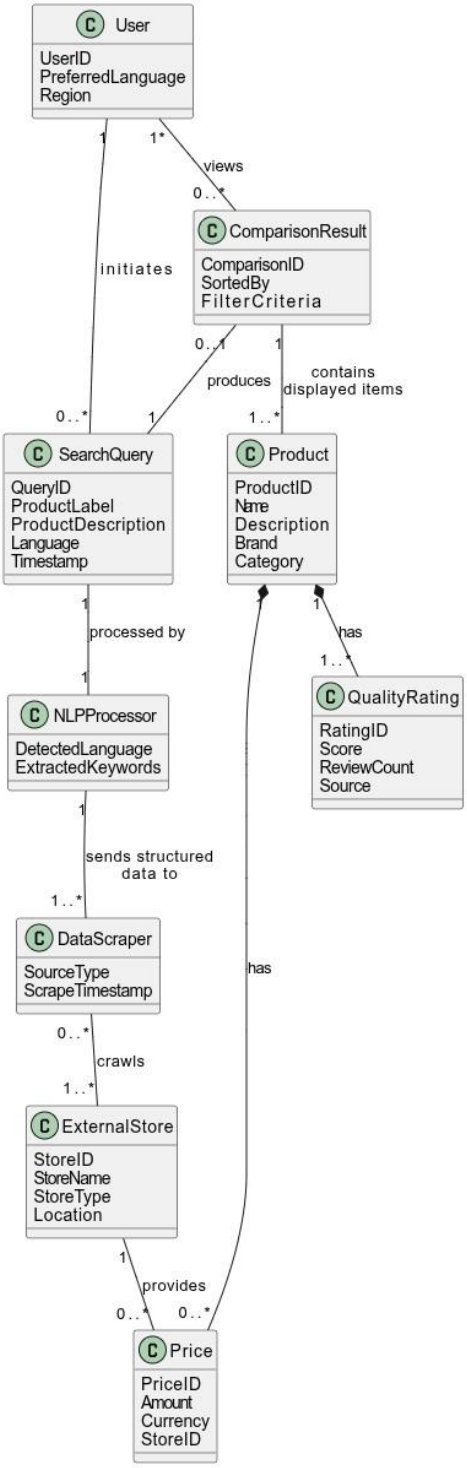

Figure 3.3: Domain Diagram

Chapter 4

Software Implementation (SI)

4.1 System Definition

A focus lands first on how the system is designed, its necessary components unfolding afterward. Requirements appear next, shaped by intended operations rather than abstract ideals. Functionality follows, built into each layer instead of added later. Structure guides every choice, making clarity a steady outcome. Users connect with the system while external components such as web scrapers and natural language processors engage indirectly, shaping how information flows through separate but linked channels

4.2 Development Tools

- **Frontend:** Next.js for responsive UI and fast rendering
- **Backend:** Node.js with Express.js for API development
- **Database:** MongoDB for storing product data, cached results, and comparison information
- **Web Scraping Engine:** Puppeteer for extracting product data from dynamic websites
- **NLP Module:** Used for query normalization, translation (Urdu $\leftrightarrow$ English), and keyword extraction
- **Version Control:** Git
- **Tailwind CSS**: Makes the website look good on all devices (phones, tablets, computers)
- **Bootstrap**: Additional design components
- **JavaScript**: Makes the website interactive
- **Testing:** Manual testing for functional accuracy and failure handling

4.3 Implementation Details

The system is implemented using a modular architecture where each module corresponds directly to system use cases.

4.3.1 User Interaction Module

This module is used for mostly all interactions users will have with this system. The users may type product questions, browse result lists, pick the products they want, use filters and see products being compared. User input is checked so that empty or incorrect searches are not

allowed. The module is directly for features like *searching product*, *viewing* what is inside a product, and *filtering* or sorting the products which people want to see.

4.3.2 Query Processing (NLP) Module

Inside this module, the description of user or name of a product is processed. It handles making input normal, identifying language and also converting Urdu questions if needed into English. Important words are taken out for increasing matching quality. When the NLP part cannot deal with the question, system asks users about rephrasing the entered input,and this follows the case covered with alternate situations for *Search a Product*.

4.3.3 Web Scraping and Data Retrieval Module

This module gathers product data from lot of online and physical store places with methods of automated web scraping. The data pulled contains prices, reviews, if the product is in stock and information about specific shop. The system is doing its best when facing some failures; it can show results from sources that worked and points out which source could not be found.

4.3.4 Product Matching and Comparison

The engine for matching and comparing products checks the data fetched, focusing on how similar items are, the cost, and with quality points. It is controlling so that comparing products are shown only if complete details are available from all sources as described earlier in product comparing cases. Products that were matched are ordered and made ready for display with the comparison that is formatted in structured way.

4.4 Data Flow

Query is sent for NLP processing and then forwarded to scraping engine. The system puts retrieved data in database for a short period, then hands it over for matching and comparison steps. Filtered, sorted and compared results are shown to user in real time and cache data is also used for seeing product details.

4.5 Summary

This chapter has described way the PQC system is made and how it is directly following its functional cases. Using different modules leads to an efficient product searching, correct comparison and a smooth way for users to interact. Joining NLP with web scraping and live filtering, PQC system completely does its work for trustworthy price and quality comparison.

Chapter 5

Software Test Documentation (STD)

5.1 Introduction

Software testing plays a key role when developing Price and Quality Comparison (PQC) system. This chapter tries to show what actions were carried out during the test phase regarding both functioning and also not functioning aspects. Software testing was done to confirm that the operations of PQC system are correct, system is reliable, easy to use and it has a high strong.

5.1.1 Test Approach

A functional and scenario-based testing approach was adopted for the PQC system. Manual testing was primarily used due to the dynamic nature of web scraping and real-time data retrieval.The testing approach included unit testing, integration testing, system testing, and user acceptance testing to ensure overall system stability.

5.2 Test Plan

The test plan defines the strategy, objectives, scope, tools, and schedule used during testing of the PQC system.

5.2.1 Testing Objectives

- To verify that all system use cases function correctly
- To ensure accurate product search and comparison results
- To validate multilingual query processing
- To confirm graceful handling of failures and invalid inputs
- To ensure fast response time and smooth user experience

5.2.2 Testing Scope

- Product search functionality
- NLP-based query processing
- Web scraping and data retrieval
- Product matching and comparison
- Filter and sort operations
- User interface responsiveness

- Error handling mechanisms

5.2.3 Testing Tools and Environment

Testing was performed using the following tools and environment:

- Operating System: Windows / Linux
- Web Browser: Google Chrome
- Frontend Framework: Next.js
- Backend Framework: Node.js with Express
- Database: MongoDB
- Web Scraping Tool: Puppeteer

5.2.4 Testing Schedule

The testing activities were carried out iteratively during the development lifecycle.

Phase	**Testing Activity**
Module Completion	Unit testing of individual modules
Feature Integration	Integration testing between modules
System Completion	System and user acceptance testing

Table 5.1: Testing Schedule

5.3 Test Cases

The following test cases were designed and executed based on the system use cases.

5.3.1 Test Case: Valid Product Search

Test Case ID	TC-FUNC-01
Requirement ID	UC-01
Objective	To verify that the system correctly displays relevant products when a valid search query is entered.
Precondition	User is on the product search interface.

Test Steps	1. Enter valid product name. 2. Click search button.
Expected Result	Relevant products along with price and quality information are displayed.
Actual Result	Product list displayed correctly with comparison data.
Status	Pass

Table 5.2: Functional Test Case: Valid Product Search

5.3.2 Test Case: Invalid Search Input

Test Case ID	TC-FUNC-02
Requirement ID	UC-01
Objective	To verify system validation for empty or invalid input.
Precondition	Search field is empty.
Expected Result	System prompts the user to enter a valid product name.
Actual Result	Validation message displayed successfully.
Status	Pass

Table 5.3: Functional Test Case: Invalid Search

5.3.3 Test Case: Multilingual Query Processing

Test Case ID	TC-NLP-01
Requirement ID	UC-01
Objective	To verify that the system processes Urdu language queries using NLP.
Expected Result	Query translated correctly and relevant results displayed.
Actual Result	Urdu query processed successfully and results shown.
Status	Pass

Table 5.4: Functional Test Case: Multilingual Query

5.3.4 Test Case: View Product Details

Test Case ID	TC-FUNC-03
Requirement ID	UC-02
Objective	To verify detailed product information display functionality.
Expected Result	Complete product details are displayed.
Actual Result	Product details displayed successfully.
Status	Pass

Table 5.5: Functional Test Case: View Product Details

5.3.5 Test Case: Product Comparison

Test Case ID	TC-COMP-01
Requirement ID	UC-03
Objective	To verify product comparison functionality across multiple sources.
Expected Result	Comparison displayed only if complete data is available.
Actual Result	Comparison displayed correctly.
Status	Pass

Table 5.6: Functional Test Case: Product Comparison

5.3.6 Test Case: Filter and Sort

Test Case ID	TC-FILT-01
Requirement ID	UC-04
Objective	To verify filtering and sorting functionality.
Expected Result	Product list updates dynamically according to selected filters.
Actual Result	Filtered and sorted results displayed correctly.
Status	Pass

Table 5.7: Functional Test Case: Filter and Sort

5.3.7 Performance Testing

Test Case ID	TC-PERF-01
Requirement ID	NFR-01
Objective	To evaluate system response time under concurrent user load.
Expected Result	Response time less than 3 seconds.
Actual Result	Average response time observed was 2.1 seconds.
Status	Pass

Table 5.8: Non-Functional Test Case: Performance

5.3.8 Reliability Testing

Test Case ID	TC-REL-01
Requirement ID	NFR-02
Objective	To verify graceful handling of external data source failure.
Expected Result	Partial results displayed and unavailable sources marked.
Actual Result	System handled failure gracefully.
Status	Pass

Table 5.9: Non-Functional Test Case: Reliability

5.3.9 Test Execution Summary

Total Test Cases	Passed	Failed
8	8	0

Table 5.10: Overall Test Execution Summary

5.4 Conclusion

The testing process confirmed that the PQC system meets its functional requirements and performs reliably under normal and exceptional conditions. All major use cases were successfully validated, demonstrating system correctness, stability, and usability.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This project successfully developed a web-based Price and Quality Comparison (PQC) system that enables users to search, compare, and analyze products from multiple online and physical stores. The system integrates NLP-based query processing, automated web scraping, and intelligent comparison techniques to deliver accurate and user-friendly results. The modular architecture, multilingual support, and real-time filtering features make the system scalable and efficient.

6.2 Future Work

Te PQC system meets its current goals, several improvements can be done in the future:

- Integration of additional online and physical store sources
- Scrape the data from Facebook and TikTok for small e-commerce stores
- User accounts with personalized preferences and history
- Development of a mobile application version
- Performance optimization for large-scale usage

These improvements would further enhance the functionality, accuracy, and scalability of the system.